

מדריך ליצירת חיבורים Credentials ו API ב n8n

מדריך מספר 2 מסדרת המדריכים בנושא n8n

כך מחברים מערכות אמיתיות לאוטומציה מאובטחת ועובדת

במדריך זה נבנה זרימה שימושית: קבלת ליד, אימות נתונים, כתיבה ל Google Sheets ושליחת התראה לצוות.

מה תלמדו	התוצאה
Credentials	חיבור סודי ומנוהל במקום API key קשיח.
HTTP Request	עבודה מול API גם כשאין Node ייעודי.
Webhook	קבלת אירוע ממערכת חיצונית בצורה נכונה.
אמינות	מניעת כפילויות, טיפול בכשל ובדיקת Execution.

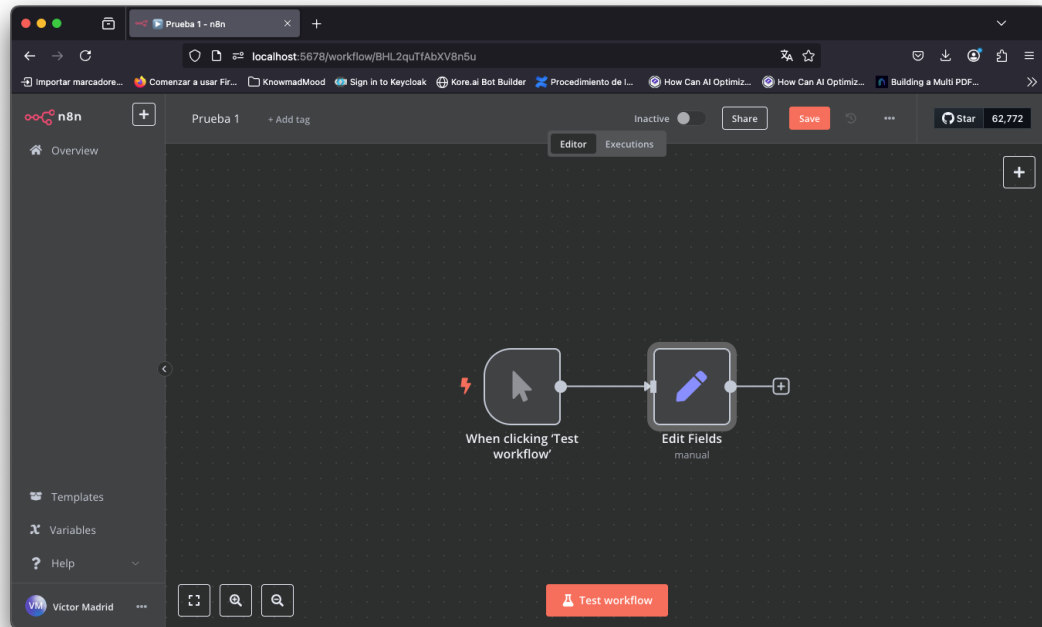
עקרונות העבודה

כל חיבור חיצוני מורכב משלושה חלקים: זהות, בקשה ותגובה. Credentials מגדירים מי אנחנו מול השירות, ה Node מגדיר מה מבקשים וה Output הוא המידע שממשיכים איתו בפלאו.

מושג	מה הוא עושה	דוגמה
Credential	שומר אמצעי הזדהות לשימוש חוזר.	OAuth לחשבון Google או API key לשירות.
Node	מבצע פעולה אחת מול השירות.	Create Row, Send Message או Get Many.
API	הממשק שהמערכת חושפת לתוכנה אחרת.	POST ליצירת ליד ב CRM.
Webhook	API הפוך: המערכת החיצונית שולחת אליכם אירוע.	טופס חדש שולח נתונים ל n8n.

בחירת דרך החיבור

אם קיים	בחרו	למה
Node ייעודי	Node של השירות	הדרך המהירה והקריאה ביותר.
API מתועד אך אין Node	HTTP Request	מאפשר כל endpoint ותומך באימות.
מערכת שולחת אירועים	Webhook Trigger	מונע polling מיותר ומגיב מיד.
אותו חיבור חוזר	Credential שמור	לא חושף סודות בכל Node.



Credentials בצורה נכונה

Credential הוא נכס אבטחה. יוצרים אותו דרך Credentials או ישירות מתוך שדה Credential ב Node. תנו לו שם לפי מערכת וסביבה, למשל Google Sheets Production או HubSpot Sandbox.

שיטת אימות	מתי משתמשים	מה נדרש
OAuth2	התחברות לחשבון משתמש או שירות שתומך בהרשאות.	Client ID, Client Secret ולעיתים Redirect URL.
API key	שירות שמספק מפתח קבוע.	מפתח, ולעיתים שם Header או Query parameter.
Bearer token	API שמקבל token ב Authorization header.	Token עם תוקף ו scopes מתאימים.
Basic Auth	שירותים ישנים או פנימיים.	שם משתמש וסיסמה, רק על HTTPS.

- לא מדביקים key בתוך Expression, Code, URL או טקסט של Node.
- יוצרים Credential נפרד ל Test ול Production. כך אי אפשר בטעות לכתוב לנתוני אמת בזמן פיתוח.
- נותנים ל token את ההרשאות המינימליות הדרושות. Read only כאשר אין צורך בכתיבה.
- כאשר משתפים workflow, בודקים מי יכול לראות או להשתמש ב Credentials הקשורים אליו.

בניית פרויקט מעשי

התרחיש: טופס באתר שולח ליד ל ח8n. הפלאו מאמת email, בודק אם אותו ליד כבר קיים, מוסיף שורה ל Google Sheets ושולח התראה ב Slack.

שלב 1 Webhook Trigger

1. צרו Webhook Trigger ובחרו POST.
2. בזמן פיתוח העתיקו את Test URL והגדירו את הטופס לשלוח אליו נתון דמה.
3. בדקו שה JSON שנכנס כולל source | name, email, company.
4. לאחר הבדיקה הפעילו את ה workflow והחליפו במקור את הכתובת ל Production URL.

שלב 2 ניקוי ואימות

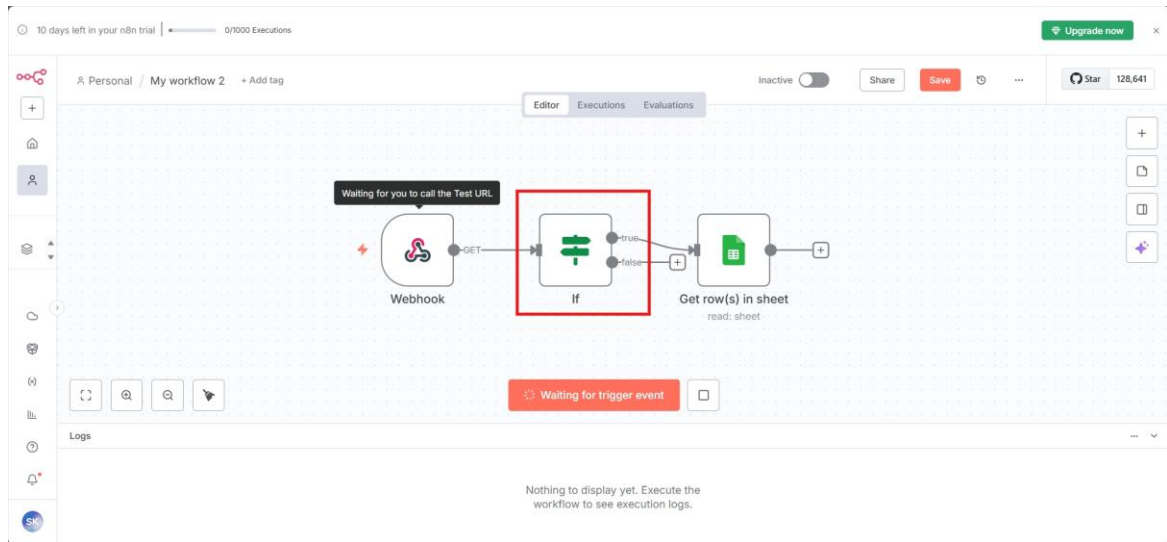
הוסיפו Edit Fields כדי לשמור רק את השדות הדרושים וליצור ערכים עקביים. לאחריו הוסיפו if: אם email חסר או אינו תקין, חברו את false לנתיב שמסיים או מתעד שגיאה. אל תשלחו רשומה ריקה ליעד.

שלב 3 Google Sheets

צרו Credential Google Sheets OAuth2 דרך ה Node. בחרו Sheet | Resource | Operation של הוספת שורה. גררו את הערכים מה INPUT לשדות המתאימים: name, createdAt | email, company. אל תקלידו ערכים ידניים בשדות דינמיים.

שלב 4 התראה

הוסיפו Slack או Gmail. בנו הודעה קצרה עם שם, אימייל וקישור לרשומה. השתמשו ב Expression, למשל `{{json.email$}}`, כדי שההודעה תכיל את ערך הריצה ולא טקסט קבוע.



HTTP Request כאשר אין Node מוכן

HTTP Request הוא Node כללי לביצוע API calls. הוא מתאים רק לאחר שמבינים את התיעוד של השירות: כתובת, method, authentication, headers, body והתגובה הצפויה.

שדה	מה מגדירים	דוגמה
Method	סוג הפעולה.	GET לקריאה, POST ליצירה, PATCH לעדכון.
URL	Endpoint מלא וללא key בתוך הכתובת.	<code>https://api.example.com/v1/leads</code>
Authentication	Credential, Bearer predefined או credential.	Predefined Credential Type כאשר קיים.
Headers	מטא מידע לבקשה.	<code>Content-Type: application/json</code>

Body	JSON שנשלח בפעולות כתיבה.	{ "email": "{{ \$json.email }}" }
Response	בדיקה מה חזר לפני מעבר לשלב הבא.	id, status, error או data.

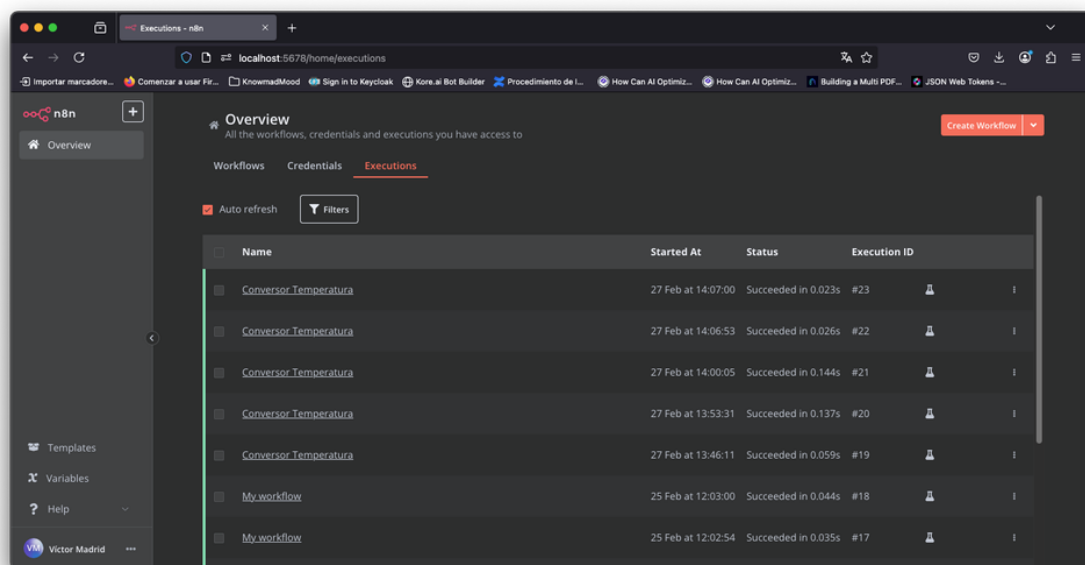
- התחילו ב GET פשוט כדי לבדוק חיבור לפני POST או DELETE.
- כאשר ה API תומך ב idempotency key, השתמשו בו בפעולות שיוצרות נתונים.
- אל תבחרו Ignore Response Code כפתרון קבוע; הוא עלול להסתיר כשל אמיתי.

מיפוי נתונים ובדיקת תגובות

בחלונות INPUT אפשר לראות מה הגיע מה Node הקודם. גררו ערך לשדה כדי ליצור Expression אוטומטית. כאשר הפלאו כולל ענפים, אפשר להפנות ל Node בשם מפורש, אך קודם בדקו שהערך באמת קיים בכל מסלול.

בדיקה	מה לחפש	פעולה מתקנת
INPUT	האם השדה קיים ובסוג הנכון.	תקנו את Edit Fields או את ה Expression.
OUTPUT	האם הפעולה הצליחה ומהו id שנוצר.	שמרו id להמשך, לא רק הודעת success.
HTTP status	2xx הצלחה; 4xx בקשה או הרשאה; 5xx שרת.	בדקו body של השגיאה לפני retry.
רשימה	כמה items חוזרים.	השתמשו ב Loop Over Items אם צריך לעבד כל פריט.

סיכון	הגנה מומלצת
ליד נכנס פעמיים	בדיקת email או eventId לפני יצירת רשומה.
rate limit או Timeout	Loop במנות ו Wait; retry מוגבל לאחר בדיקת idempotency.
Webhook ציבורי	אימות header או חתימה, והגבלת הקלט הצפוי.
כשל לא נראה	Error Trigger שמדווח ל Slack או Email עם Execution ID.
סוד דלף	החלפת credential, ביטול token והרצת audit לפני חזרה לייצור.



איור 3: מסך Executions הוא מקור האמת לריצות, שגיאות ומשך ביצוע.

מקור צילום המסך: <https://www.enmilocalfunciona.io/ejemplo-de-n8n-uso-del-nodo-code>

צ'ק ליסט לפני הפעלה

- ☐ Credential נשמר תחת שם ברור ואינו מוטמע ב Node או בקוד.
- ☐ נבדקו Test URL ו Production URL בנפרד.
- ☐ נבדקו קלט תקין, קלט חסר, כפילות וכשל של השירות החיצוני.
- ☐ כל פעולת כתיבה כוללת מניעת כפילות או מזהה אירוע.
- ☐ מוגדר מסלול Error עם בעלים והתראה.
- ☐ נבדק Execution מלא עם מידע אמיתי ומבוקר.
- ☐ ה workflow הופעל רק לאחר שמירה ובדיקת ההרשאות.

מקורות להעמקה

/Data mapping: <https://docs.n8n.io/data/data-mapping/data-mapping-ui>

/Executions: <https://docs.n8n.io/workflows/executions/all-executions>

HTTP Request: <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest>

/Security audit: <https://docs.n8n.io/hosting/securing/security-audit>